TypeScript para Pythonistas

Preparación Técnica — Toku

Stack objetivo: `TypeScript + Vue.js + FastAPI`
Foco: Entender TS viniendo de Python
Nivel: Intermedio Python → Básico-Intermedio TS

Temas

- Tipado estático y estructural
- Funciones y arrow functions
- Async/Await y Promises
- Generics
- Módulos e imports
- Vue.js Composition API
- 5 Ejercicios con soluciones

Marzo 2026 — Estudio Personal

Índice

1. Tipado: De Python a TypeScript **2**
1.1. Tipos primitivos 2
1.2. Interfaces y Types 2
1.3. Union e Intersection 3

2. Funciones y Arrow Functions **4**

3. Async/Await y Promises **4**
3.1. Comparación directa 5
3.2. Promises en paralelo 6

4. Generics **6**

5. Módulos e Imports **7**

6. Vue.js 3 — Composition API **9**
6.1. Props y Emits (comunicación entre componentes) 10

7. Tabla de Equivalencias Python → TypeScript **11**

8. Ejercicios Prácticos **11**

9. Cheatsheet: TypeScript en Una Página **15**

Tipado: De Python a TypeScript

Tipos primitivos

La idea clave

Python tiene **type hints** opcionales (`int`, `str`...). TypeScript tiene tipado **estático obligatorio** (en la práctica). Si no pones tipo, TS infiere; si infiere `any`, es una señal de alerta.

Python

```python
# Python
nombre: str = "Fabian"
edad: int = 25
precio: float = 9990.0
activo: bool = True
nada: None = None
cualquier_cosa: any = 42    # evitar
```

TypeScript

```typescript
// TypeScript
const nombre: string = "Fabian";
const edad: number = 25;          // number = int y float
const precio: number = 9990.0;
const activo: boolean = true;
const nada: null = null;
const indefinido: undefined = undefined;
let cualquier_cosa: any = 42;    // evitar: apaga el tipado
```

Ojo

En TS `number` cubre tanto enteros como flotantes (no hay `int` y `float` separados).

Interfaces y Types

Equivalente a las `dataclass` o `TypedDict` de Python:

Python

```python
from dataclasses import dataclass
from typing import TypedDict, Optional

@dataclass
class Cliente:
    id: str
    nombre: str
    email: str
    telefono: Optional[str] = None

class ClienteDict(TypedDict):
    id: str
    nombre: str
    email: str
```

TypeScript

```typescript
// Interface (preferido para objetos/formas)
interface Cliente {
  id: string;
  nombre: string;
  email: string;
  telefono?: string;     // ? = opcional (equivale a Optional[str])
  readonly createdAt: Date; // readonly = no mutable
}

// Type alias (mas flexible, puede ser union/intersection)
type Estado = "activo" | "inactivo" | "pendiente"; // union literal

type ClienteResumen = Pick<Cliente, "id" | "nombre">; // solo esos
    campos
type ClienteParcial = Partial<Cliente>;                // todos
    opcionales
```

Tip

interface vs type: Usa `interface` para describir objetos y clases. Usa `type` para uniones, intersecciones y alias de tipos complejos. En la práctica son muy similares para casos simples.

Union e Intersection

Python

```python
from typing import Union

def procesar(valor: Union[str, int]) -> str:
    return str(valor)

# Python 3.10+
def procesar(valor: str | int) -> str:
    return str(valor)
```

TypeScript

```typescript
// Union: puede ser uno U otro
type IdOrName = string | number;

function procesar(valor: string | number): string {
  return String(valor);
}

// Intersection: debe cumplir ambos
interface Timestamps { createdAt: Date; updatedAt: Date; }
interface ClienteBase { id: string; nombre: string; }

type ClienteCompleto = ClienteBase & Timestamps;
// ahora necesita id, nombre, createdAt Y updatedAt
```

Funciones y Arrow Functions

Python

```python
# Funcion normal
def saludar(nombre: str, saludo: str = "Hola") -> str:
    return f"{saludo}, {nombre}!"

# Lambda
cuadrado = lambda x: x ** 2

# Funcion que recibe funcion
def aplicar(fn: callable, valor: int) -> int:
    return fn(valor)
```

TypeScript

```typescript
// Funcion normal (tipada)
function saludar(nombre: string, saludo: string = "Hola"): string {
  return `${saludo}, ${nombre}!`;  // template literals con backtick
}

// Arrow function (equivale a lambda pero mas poderosa)
const cuadrado = (x: number): number => x ** 2;

// Arrow sin llaves = return implicito (solo expresiones simples)
const doble = (x: number) => x * 2;

// Tipo de una funcion
type Transformador = (valor: number) => number;
const aplicar = (fn: Transformador, valor: number): number => fn(valor)
    ;

// Parametros rest (equivale a *args)
const sumar = (...numeros: number[]): number =>
  numeros.reduce((acc, n) => acc + n, 0);
```

Tip

Las arrow functions en TS/JS tienen una diferencia crucial con `def`: el valor de `this` es léxico (del contexto donde se define). Esto importa en Vue.js y clases.

Async/Await y Promises

Comparación directa

Python

```python
import asyncio
import httpx

async def obtener_cliente(id: str) -> dict:
    async with httpx.AsyncClient() as client:
        response = await client.get(f"/customers/{id}")
        return response.json()

async def main():
    cliente = await obtener_cliente("cust_123")
    print(cliente["nombre"])

asyncio.run(main())
```

TypeScript

```typescript
// Promise<T> = equivale a Coroutine/Awaitable en Python
async function obtenerCliente(id: string): Promise<Cliente> {
  const response = await fetch(`/customers/${id}`);
  if (!response.ok) throw new Error("Cliente no encontrado");
  return response.json() as Promise<Cliente>;
}

// Uso
async function main() {
  const cliente = await obtenerCliente("cust_123");
  console.log(cliente.nombre);
}

// Manejo de errores (equivale a try/except)
async function seguro() {
  try {
    const cliente = await obtenerCliente("cust_123");
  } catch (error) {
    console.error("Error:", error);
  } finally {
    console.log("Siempre se ejecuta");
  }
}
```

Promises en paralelo

Python

```python
import asyncio

# Ejecutar en paralelo
clientes = await asyncio.gather(
    obtener_cliente("c1"),
    obtener_cliente("c2"),
    obtener_cliente("c3"),
)
```

TypeScript

```typescript
// Promise.all = asyncio.gather
const [c1, c2, c3] = await Promise.all([
  obtenerCliente("c1"),
  obtenerCliente("c2"),
  obtenerCliente("c3"),
]);

// Promise.allSettled = no falla si uno falla
const resultados = await Promise.allSettled([
  obtenerCliente("c1"),
  obtenerCliente("inexistente"),  // este puede fallar
]);
resultados.forEach(r => {
  if (r.status === "fulfilled") console.log(r.value);
  else console.error(r.reason);
});
```

Generics

Python

```python
from typing import TypeVar, Generic, List

T = TypeVar('T')

class Repositorio(Generic[T]):
    def __init__(self):
        self._items: List[T] = []

    def agregar(self, item: T) -> None:
        self._items.append(item)

    def obtener(self, i: int) -> T:
        return self._items[i]

repo_clientes: Repositorio[Cliente] = Repositorio()
```

TypeScript

```typescript
// Generic con <T> (sintaxis directa, sin TypeVar)
class Repositorio<T> {
  private items: T[] = [];

  agregar(item: T): void {
    this.items.push(item);
  }

  obtener(i: number): T {
    return this.items[i];
  }
}

const repoClientes = new Repositorio<Cliente>();

// Funciones genericas
function primero<T>(lista: T[]): T | undefined {
  return lista[0];
}

// Con constraint (equivale a TypeVar con bound=)
function obtenerNombre<T extends { nombre: string }>(entidad: T):
    string {
  return entidad.nombre;
}

// Tipos de utilidad genericos (muy usados en APIs)
type ApiResponse<T> = {
  data: T;
  ok: boolean;
  error?: string;
};

type ClienteResponse = ApiResponse<Cliente>;
type ListaResponse = ApiResponse<Cliente[]>;
```

Módulos e Imports

Python

```python
# cliente.py
class Cliente:
    pass

def crear_cliente(data: dict) -> Cliente:
    pass

# main.py
from cliente import Cliente, crear_cliente
import cliente as mod
from cliente import *  # evitar
```

TypeScript

```typescript
// cliente.ts
export interface Cliente {
  id: string;
  nombre: string;
}

export function crearCliente(data: Partial<Cliente>): Cliente {
  return { id: crypto.randomUUID(), nombre: data.nombre ?? "" };
}

export default class ClienteService {}  // export default = un solo default

// main.ts
import { Cliente, crearCliente } from "./cliente";  // named exports
import ClienteService from "./cliente";              // default export
import * as clienteMod from "./cliente";             // namespace import
import type { Cliente } from "./cliente";            // solo el tipo (no JS)
```

Tip

`import type` es importante en TypeScript: importa solo la definición de tipo, sin incluir el código JavaScript en el bundle. Úsalo cuando solo necesitas el tipo para anotaciones, no el valor en runtime.

Vue.js 3 — Composition API

Vue.js

```
<script setup lang="ts">
// <script setup> es la forma moderna (Vue 3.2+)
// Equivale a defineComponent() pero mas limpio

import { ref, reactive, computed, watch, onMounted } from 'vue';

// ref() = variable reactiva simple (primitivos)
// Acceso con .value en JS, pero sin .value en el template
const count = ref<number>(0);
const nombre = ref<string>('');

// reactive() = objeto reactivo completo
const estado = reactive({
  cargando: false,
  error: null as string | null,
  clientes: [] as Cliente[],
});

// computed() = propiedad derivada (equivale a @property en Python)
const nombreMayusculas = computed(() => nombre.value.toUpperCase());
const hayClientes = computed(() => estado.clientes.length > 0);

// watch() = observar cambios (como un setter con efecto secundario)
watch(nombre, (nuevoValor, valorAnterior) => {
  console.log(`Cambio: ${valorAnterior} -> ${nuevoValor}`);
});

// onMounted() = se ejecuta cuando el componente monta (como useEffect
    [] en React)
onMounted(async () => {
  estado.cargando = true;
  try {
    estado.clientes = await obtenerClientes();
  } finally {
    estado.cargando = false;
  }
});

// Funciones normales (automaticamente disponibles en el template)
function incrementar() {
  count.value++;
}

async function cargarCliente(id: string) {
  const cliente = await obtenerCliente(id);
  estado.clientes.push(cliente);
}
</script>

<template>
  <!-- count sin .value en el template -->
  <p>{{ count }}</p>
  <button @click="incrementar">+1</button>

  <!-- v-if, v-for son equivalentes a if/for de Jinja2 -->
  <div v-if="estado.cargando">Cargando...</div>
  <ul v-else>
    <li v-for="cliente in estado.clientes" :key="cliente.id">
      {{ cliente.nombre }}
    </li>
  </ul>
```

Props y Emits (comunicación entre componentes)

Vue.js

```
<script setup lang="ts">
// Props (datos que entran al componente, como argumentos)
interface Props {
  clienteId: string;
  readonly?: boolean;
}
const props = defineProps<Props>();

// Emits (eventos que salen del componente)
const emit = defineEmits<{
  'cliente-guardado': [cliente: Cliente];
  'cancelado': [];
}>();

// Uso:
emit('cliente-guardado', { id: '1', nombre: 'Juan', email: 'j@j.com' })
    ;
</script>

<template>
  <!-- :clienteId pasa prop dinamica, clienteId="literal" es string
      literal -->
  <ChildComponent :cliente-id="props.clienteId" @cliente-guardado="
      manejar" />
</template>
```

Tabla de Equivalencias Python → TypeScript

Concepto	Python	TypeScript
Tipo string	`str`	`string`
Tipo entero	`int`	`number`
Tipo flotante	`float`	`number`
Tipo booleano	`bool`	`boolean`
Sin tipo / desconocido	`Any`	`any` / `unknown`
Opcional	`Optional[T]`	`T | undefined` / `T?`
Lista	`List[T]`	`T[]` / `Array<T>`
Diccionario	`Dict[str, T]`	`Record<string, T>`
Tupla	`Tuple[int, str]`	`[number, string]`
Clase de datos	`@dataclass`	`interface` / `class`
Herencia	`class B(A)`	`class B extends A`
Mixin	`class C(A, B)`	`class C extends A implements B`
TypeVar/Generic	`TypeVar, Generic[T]`	`<T>`
Lambda	`lambda x: x+1`	`(x) =>x+1`
Comprensión lista	`[x*2 for x in l]`	`l.map(x =>x*2)`
Filtro	`[x for x in l if x>0]`	`l.filter(x =>x>0)`
Reduce	`reduce(fn, l, 0)`	`l.reduce(fn, 0)`
Formato string	`f"hola {x}"`	`` `hola $x` ``
Null	`None`	`null` / `undefined`
Asyncio gather	`asyncio.gather()`	`Promise.all()`
Try/except/finally	`try/except/finally`	`try/catch/finally`
Decorador	`@decorator`	`@Decorator()` (limitado)
Módulo import	`from mod import fn`	`import { fn } from './mod'`

Ejercicios Prácticos

Ejercicio 1 — Tipado básico

Tienes esta función en Python. Pásala a TypeScript con tipado correcto:

```python
def calcular_monto_con_interes(
    monto: float,
    tasa: float = 0.05,
    periodos: int = 12
) -> dict:
    total = monto * ((1 + tasa) ** periodos)
    return {"original": monto, "total": total, "interes": total - monto
        }
```

Solución 1

```
interface ResultadoInteres {
  original: number;
  total: number;
  interes: number;
}

function calcularMontoConInteres(
  monto: number,
  tasa: number = 0.05,
  periodos: number = 12
): ResultadoInteres {
  const total = monto * Math.pow(1 + tasa, periodos);
  return { original: monto, total, interes: total - monto };
}
```

Ejercicio 2 — Async/Await con manejo de error

Escribe una función TypeScript `obtenerClienteSeguro(id: string)` que llame a una API, devuelva el cliente si existe, o `null` si falla (sin lanzar error al caller).

Solución 2

```
async function obtenerClienteSeguro(id: string): Promise<Cliente | null
    > {
  try {
    const response = await fetch(`https://api.trytoku.com/customers/${
        id}`, {
      headers: { Authorization: `Bearer ${process.env.TOKU_API_KEY}` },
    });
    if (!response.ok) return null;
    return response.json() as Promise<Cliente>;
  } catch {
    return null;
  }
}
```

Ejercicio 3 — Generics con restriccion

Crea una función genérica `paginar<T>` que reciba un arreglo de T, un número de página y tamaño de página, y devuelva `{ items: T[], total: number, pagina: number, totalPaginas: number }`.

Solución 3

```
interface Pagina<T> {
  items: T[];
  total: number;
  pagina: number;
  totalPaginas: number;
}

function paginar<T>(lista: T[], pagina: number, tamano: number = 10):
    Pagina<T> {
  const total = lista.length;
  const totalPaginas = Math.ceil(total / tamano);
  const inicio = (pagina - 1) * tamano;
  const items = lista.slice(inicio, inicio + tamano);
  return { items, total, pagina, totalPaginas };
}
```

Ejercicio 4 — Vue.js Composable

Crea un composable `useClientes()` que exponga: `clientes` (lista reactiva), `cargando` (boolean), `cargar()` (función async) y `total` (computed).

Solución 4

```
// useClientes.ts
import { ref, computed } from 'vue';

export function useClientes() {
  const clientes = ref<Cliente[]>([]);
  const cargando = ref(false);
  const error = ref<string | null>(null);

  const total = computed(() => clientes.value.length);

  async function cargar() {
    cargando.value = true;
    error.value = null;
    try {
      const resp = await fetch('/api/customers');
      clientes.value = await resp.json();
    } catch (e) {
      error.value = 'Error al cargar clientes';
    } finally {
      cargando.value = false;
    }
  }

  return { clientes, cargando, error, total, cargar };
}
```

Ejercicio 5 — Tipos de utilidad Toku

Dado el modelo de Toku, crea tipos TypeScript para: `Invoice` (deuda), `InvoiceCreate` (para crear, sin id ni fechas automáticas), `InvoiceEstado` (union literal de estados posibles).

Solución 5

```typescript
type InvoiceEstado = "pending" | "paid" | "failed" | "void" | "partially_paid";

interface Invoice {
  id: string;
  customerId: string;
  productId: string;
  externalId?: string;
  amount: number;            // en centavos o moneda base
  currency: "CLP" | "MXN" | "BRL";
  estado: InvoiceEstado;
  dueDate: Date;
  paidAt?: Date;
  createdAt: Date;
  updatedAt: Date;
}

// Para crear: sin campos autogenerados
type InvoiceCreate = Omit<Invoice, "id" | "estado" | "paidAt" | "createdAt" | "updatedAt">;

// Para actualizar: todo opcional
type InvoiceUpdate = Partial<Pick<Invoice, "externalId" | "amount" | "dueDate">>;
```

Cheatsheet: TypeScript en Una Página

Tipos básicos

```
string, number, boolean
null, undefined, void
any, unknown, never
T[], Array<T>
[T, U]            // tupla
Record<K, V>      // diccionario
```

Interfaces

```
interface A { x: number; y?: string }
type B = A & { z: boolean }  // interseccion
type C = A | B               // union
type D = Partial<A>          // todos opcionales
type E = Required<A>         // todos requeridos
type F = Pick<A, "x">        // solo algunos campos
type G = Omit<A, "y">        // sin algunos campos
```

Funciones

```
function f(x: number): string {}
const f = (x: number): string => {};
const f = (x = 0) => x;      // default
const f = (...xs: number[]) => {}; // rest
```

Async

```
async function f(): Promise<T> {
  const x = await promesa;
}
await Promise.all([p1, p2]);
await Promise.allSettled([p1, p2]);
```

Generics

```
function id<T>(x: T): T { return x; }
class Repo<T> { items: T[] = []; }
function fn<T extends {id: string}>(x: T) {}
```

Vue Composition API

```
const x = ref<number>(0);  // reactivo
x.value++;                 // acceso con .value
const obj = reactive({});  // objeto reactivo
const c = computed(() => x.value * 2);
watch(x, (nuevo, viejo) => {});
onMounted(async () => { ... });
```

Template Vue

```
{{ variable }}
:prop="valor"     // prop dinamica
@click="fn"       // evento
v-if="cond"
v-for="x in xs" :key="x.id"
v-model="ref"     // two-way binding
```

Patrones útiles

```
x ?? "default"          // nullish coalescing
x?.propiedad            // optional chaining
x!.propiedad            // non-null assertion
x as Tipo               // type assertion
typeof x === "string"   // type guard
```